

PROJECT AND TEAM INFORMATION

Project Title

Distributed LSM-Tree Database Engine

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>DataStellars</i> <i>T042</i>
Team member 1 (Team Lead)	<i>Jain, Atishay – 24021451</i> <i>jain.atishay44@gmail.com</i> 
Team member 2	<i>Kathait, Sarthak – 24021911</i> <i>sarthakkathait.me@gmail.com</i> 
Team member 3	<i>Singh, Vinayak – 240211006</i> <i>vinayaksinghddn@gmail.com</i> 
Team member 4	<i>Dangwal, Suhani – 24022828</i> <i>suhanidangwal2005@gmail.com</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Modern distributed databases are forced to trade off high write throughput against strict consistency. Traditional storage engines rely heavily on the Linux kernel's page cache and blocking system calls (`read/write`), which stall CPU threads and introduce unpredictable tail-latency spikes when flushing to disk. In distributed deployments, network partitions can additionally cause split-brain behaviour and stale reads, and engines are notoriously hard to validate against edge-case hardware or network failures until they break in production. This matters because low-latency, strongly consistent storage underpins virtually every modern online system – ledgers, session stores, real-time analytics – all of which depend on a layer that never silently loses or reorders data. We address this by tightly coupling a hardware-sympathetic async I/O engine with a proven consensus algorithm and continuous automated failure testing.

State of the Art / Current solution (1 pt)

Today's LSM-tree engines, such as RocksDB and Cassandra's storage layer, achieve high write throughput using MemTables, WALs, and background compaction, but most still rely on standard POSIX blocking I/O and the OS page cache, limiting their ability to saturate modern NVMe hardware. On the distributed side, systems like CockroachDB and TiKV pair an LSM core with Raft to provide linearizable reads/writes, leader election, and log-replication fault tolerance. Resilience validation is typically handled separately via Jepsen or Chaos Mesh, run as periodic test suites rather than continuously integrated. What's missing is the vertical integration we propose: kernel-level async I/O (`io_uring/0_DIRECT`) combined with Raft guarantees and chaos testing built directly into CI/CD.

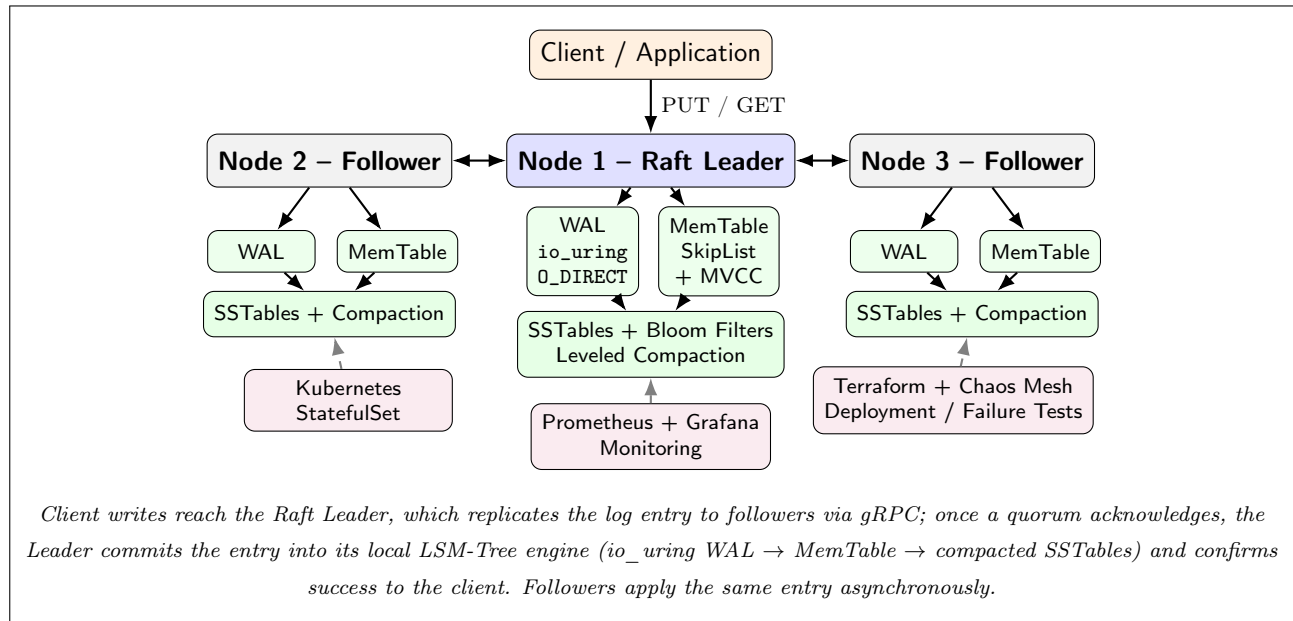
Project Goals and Milestones (2 pts)

Our goal is a functional, production-grade distributed key-value store demonstrating measurable P99 latency gains from async I/O while guaranteeing linearizable consistency and automatic fault recovery. Organized into eight weekly milestones: **W1** – provision Terraform infra, deploy K8s StatefulSets, define gRPC/Protobuf schemas; **W2** – concurrent SkipList MemTable with MVCC timestamping; **W3** – `io_uring` WAL append and `0_DIRECT` SSTable flush; **W4** – multithreaded Leveled Compaction, sparse indexing, Bloom filters; **W5** – Raft leader election and peer discovery; **W6** – Raft log replication and quorum commit; **W7** – Prometheus/Grafana telemetry and Chaos Mesh fault injection in CI/CD; **W8** – stabilize against Jepsen/Chaos findings, benchmark `io_uring` vs POSIX, finalize docs.

Project Approach (3 pts)

We will implement the storage core in C++20 or Rust for memory safety and zero-cost abstractions suitable for low-level kernel I/O. At the single-node layer, writes are first appended to a WAL via `liburing` and `0_DIRECT`, bypassing the page cache, then inserted into a concurrent SkipList MemTable tagged with MVCC timestamps; full MemTables flush to immutable SSTables indexed with sparse indexes and Bloom filters, with a background pool performing Leveled Compaction. At the cluster layer, nodes communicate over gRPC/Protobuf and use Raft for leader election, randomized heartbeats, and quorum-based log replication. Infrastructure is provisioned via Terraform (VPCs, high-IOPS storage) on Kubernetes StatefulSets with headless services for deterministic peer discovery. Prometheus/Grafana provide real-time telemetry, while Chaos Mesh and Jepsen are embedded in a GitHub Actions CI/CD pipeline to continuously simulate packet loss, disk degradation, and node crashes.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

By the end of the 8-week cycle we will deliver: (1) a functional distributed key-value store with high-throughput async writes and linearizable reads across a 3-node Raft cluster; (2) benchmark data comparing *io_uring*/*O_DIRECT* against POSIX blocking I/O, quantifying P99 latency reduction; (3) an automated GitOps pipeline validating every commit against simulated network partitions, disk-I/O degradation, and node crashes via Chaos Mesh/Jepsen; and (4) real-time telemetry (Prometheus/Grafana) covering storage metrics (WAF, IOPS) and cluster health, plus full deployment documentation.

Assumptions

We assume access to a Linux 5.1+ environment (bare-metal or cloud VM) with *liburing* support and NVMe-class storage exposing measurable IOPS gains under *O_DIRECT*. We assume a fixed 3-node cluster for the initial Raft implementation, with each of the 4 team members contributing 1–2 hours daily, and sufficient cloud quota for Terraform-provisioned VPCs, Kubernetes StatefulSets, and CI-integrated chaos testing (Chaos Mesh, Jepsen).

References

- Ongaro & Ousterhout, In Search of an Understandable Consensus Algorithm (Raft), USENIX ATC 2014.
- O’Neil et al., The Log-Structured Merge-Tree, Acta Informatica 1996.
- Linux *io_uring* docs (kernel.dk) RocksDB Wiki (github.com/facebook/rocksdb)
- CockroachDB Docs (cockroachlabs.com) Chaos Mesh (chaos-mesh.org) Jepsen (jepsen.io)